KÜHNE LOGISTICS UNIVERSITY

Management Science
& Operations Research

DO NOT EAT THE POISON

Reinforcement Learning Agent for the Game of Chomp

SARSA vs Q-Learning · 4×5 Board · MDP Formulation

Ankita Kumari | Aya Chguiri | Carter Cunden | Ngoc Anh Hoang | Zhushan He

Prof. Dr. Arne Heinold

arne.heinold@klu.org

Presentation Agenda

01

Intro to The Game

What is Chomp and why it is an ideal RL testbed

02

MDP Formulation

States · Actions · Rewards — reducing 1M+ states to 126

03

Algorithms

SARSA (on-policy) vs Q-Learning (off-policy) + backward pass

04

Results

Win rates: 99.4% vs Random, 100% vs Heuristic, …

05

Conclusion

Key takeaways and next steps

01 The Game of Chomp

How Chomp Works

Chomp is a deterministic, perfect-information sequential game

The Board

4 rows × 5 columns of chocolate squares

Each Turn

Bite any square (r,c) - removes it and everything below-right

The Poison

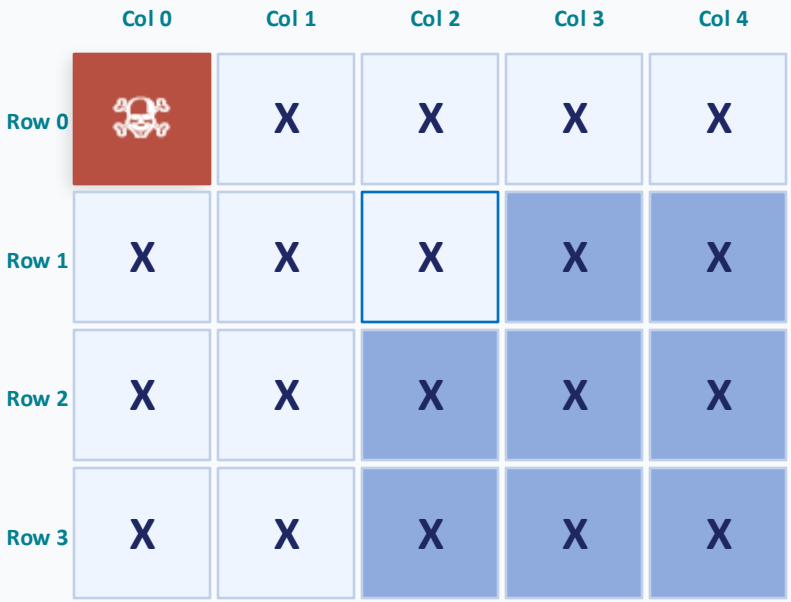
Top-left square (0,0) is poisoned - avoid it!

Losing

This is a misère game: The player FORCED to eat (0,0) loses

First Mover

First player has a theoretical winning strategy - but discovering it requires strategic reasoning or learning



Example: biting (1,2) removes shaded region →

Project Objective

1

Modeling

- Model Chomp as a Markov Decision Process
- Define states, actions, transitions, rewards
- Reduce state space to 126 valid configurations

2

Learning Approach

- Train an RL agent using SARSA and Q-learning
- Learn optimal policy through interaction
- Use backward updates due to sparse reward

3

Benchmarking

- Compare against:

  Random policy

  Rule-based heuristic

Ceiling (Exact Solver)

- Evaluate convergence and win rate

KÜHNE LOGISTICS UNIVERSITY

02

Markov Decision Process

States, Actions, Transitions, Rewards

Markov Decision Process

The complete (S, A, T, R, γ) specification

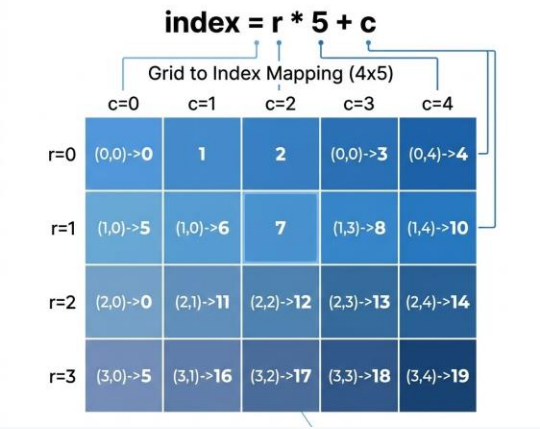
$2^{20} > 1{,}000{,}000$

possible binary grids

126

valid tuple-states

99.988% reduction



S

States

Tuple (x_0,x_1,x_2,x_3) — row lengths. Non-increasing constraint: $x_0 \geq x_1 \geq x_2 \geq x_3$ enforces valid boards. C(9,4) = 126.

- **Naive Approach:** 4x5 Binary Grid = 2^20 (>1,000,000) states
- **Game Physics:** Biting creates a strict "staircase" shape
- **Tuple Encoding:** Count remaining squares per row -> (x0, x1, x2, x3)
- **Constraint:** x0 >= x1 >= x2 >= x3
- **Result:** Only 126 valid states (99.988% reduction)

A

Actions

Any (r,c) with a chocolate square, excluding poison at (0,0). 1–19 legal moves depending on state.

- **Action Space (A):** Selecting any valid (r,c) coordinate.
- **Rule:** Strictly avoid the poison at (0,0).
- **Gymnasium API:** Action space defined as Discrete(20). **(chomp_env.py)**
- **Encoding:** Maps 2D coordinate (r, c) to a 1D index r * 5 + c (0 to 19).
- **Decoding:** The step() function parses the integer back to (r, c) before applying the bite.

T

Transitions

apply_action(s,a): sets row lengths ← min(current length, c) for rows ≥ r. Deterministic for agent; random for opponent.

- **Transition Function (T):** Deterministic board physics, but stochastic outcomes.
- **Critical Insight:** The next state P(s'|s,a) includes the random opponent's counter-move.

R

Rewards

Sparse: +1.0 on win, −1.0 on loss or illegal move, 0.0 for all intermediate moves.

- **Reward Function (R):** Extremely sparse.
- 0.0 : All intermediate moves.
- +1.0 : Win (Forcing opponent to eat poison).
- -1.0 : Loss (Eating poison or illegal move).

- **Discount Factor (Gamma):** Set to 0.9.
- *Why 0.9?* Creates a mathematical gradient to prefer the shortest path to victory.
- **The Challenge:** Sparse rewards necessitate our "Backward Pass" architecture.

KÜHNE LOGISTICS UNIVERSITY

03

Solution Methods

SARSA · Q-Learning · Backward Pass

SARSA vs Q-Learning

On-policy vs Off-policy temporal difference learning

SARSA — On-Policy	Q-Learning — Off-Policy
Learns the value of the CURRENT policy	Learns the value of the OPTIMAL policy
Update uses Q(s′, a′) — the action actually taken	Update uses max Q(s′,·) — the BEST possible action
More conservative — penalises exploratory mistakes	More aggressive — faster convergence
Backward pass propagates win/loss to ALL moves	Same backward pass interface as SARSA
Result: 99.4% vs Random · 100% vs Heuristic	Requires full training to match SARSA results
$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',\cdot) - Q(s,a)]$

The Sparse Reward Problem

Why a standard TD update fails — and how the Backward Pass fixes it

The Problem

Chomp gives 0.0 reward for EVERY intermediate move. Only the final move yields +1 or −1.

A standard online update would only learn from the last move. Every earlier decision stays at zero — the agent never learns why it won or lost.

The Solution: Backward Pass

After each episode ends:
1. Walk the trajectory in REVERSE
2. Propagate +1 or −1 backward through every (s,a) pair
3. Every move that led to a win gets a positive update
4. Every move on a losing path gets a negative update

next_q = 0.0 for state, action, reward in reversed(trajectory): $G_{t+1} = (0,0)$

TD_{Target} = Reward + Gamma(NextQ) $G_t = r_{t+q} + \gamma G_{t+1}$

Q[state][action] ±= Alpha (TD_target - Q[state][action]) $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[G_t - Q(s_t, a_t)]$

next_q = Q[state][action] Propagate backward $G_t = Q(s_t, a_t)$

Epsilon *= epsilon_decay $\epsilon_{t+1} = \epsilon_t . decay_rate$

Training Setup

10,000 episodes · ε-greedy exploration · backward pass

HYPERPARAMETERS		
Learning rate α	0.1	*Conservative — stable convergence*
Discount γ	0.9	*Future wins valued highly*
Initial ε	1.0	*Full exploration at start*
Minimum ε	0.01	*Never fully stops exploring*
ε decay	0.995	*~1k episodes to near-exploit*
Episodes N	10,000	*Sufficient for 126-state space*

01 Reset environment

New game, no seed (opponent varies)

↓

02 Choose action

ε-greedy: explore or exploit Q-table

↓

03 Step environment

Agent move → opponent move → reward

↓

04 Record trajectory

Store (state, action, reward) for later

↓

05 Backward pass

Propagate terminal reward; decay ε

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

cmd

```
(.venv) C:\Users\Carter\OneDrive\Documents\KLU\KLU Studies\Managment Science and Research Operations\Do_not_eat_the_Poison_MDP>p
```

KÜHNE LOGISTICS UNIVERSITY

04 Evaluation & Results

How we tested · What we measured · What the numbers mean

Evaluation Design

How we ensured fair, reproducible, and meaningful results

01

Fixed Seed = Reproducibility

Every matchup uses seed=42. The i-th game in each matchup uses seed 42+i. This means anyone can re-run the code and get exactly the same numbers.

02

Greedy Policy Only

During evaluation, epsilon = 0. The agent always picks the highest Q-value action — no random exploration. This tests the LEARNED strategy, not training behaviour.

03

1,000 Games Per Matchup

1,000 games gives a ±3% margin of error at 95% confidence. Small enough to run fast, large enough to be statistically meaningful.

Three Opponents — Three Difficulty Levels

Floor · Human Benchmark · Mathematical Ceiling

FLOOR

Random Bot

HOW IT PLAYS

Picks any legal move uniformly at random on every turn.

WHY WE USE IT

Sets the performance floor. Any trained agent MUST beat this to prove it learned something. Target: >85% win rate.

Expected: ~98–99% for a well-trained agent

BENCHMARK

Heuristic Bot

HOW IT PLAYS

Uses three human logic rules: (1) Take the kill shot if available. (2) Create a square trap if possible. (3) Otherwise random.

WHY WE USE IT

Represents a human player who knows some strategy. Beating this proves the agent learned BETTER than simple heuristics.

Expected: ~85–100% shows strong learned strategy

CEILING

Exact Solver

HOW IT PLAYS

Retrograde analysis over all 126 states. Classifies every state as P-position or N-position. Always plays the mathematically optimal move.

WHY WE USE IT

The mathematical ceiling — no agent can consistently beat this unless it has also learned near-optimal play.

Expected: ~90%+ is exceptional; even 60% is impressive

Evaluation Results

1,000 games per matchup · Seed 42 · Greedy policy (ε = 0)

Agent	Opponent	Win Rate	Assessment	Verdict
SARSA	Random	**99.4%**	*Target >85% — exceeded by 14 pts*	Excellent
SARSA	Heuristic	**100%**	*Perfect — beats all human logic*	Perfect
SARSA	Exact Solver	**92.7%**	*Beats optimal play 9/10 times*	Exceptional
Q-Learning	Random	**99.1%**	*Matches SARSA — both converged*	Excellent
Q-Learning	Heuristic	**100%**	*Perfect — matches SARSA exactly*	Perfect
Q-Learning	Exact Solver	**93.4%**	*Slightly edges SARSA vs optimal play*	Exceptional

eval > evaluate.py

```python
        self.name = name
        try:
            self.q_table = np.load(q_table_path)
            if self.q_table.ndim != 2 or self.q_table.shape != (126, 20):
                raise ValueError(f"Unexpected shape: {self.q_table.shape}")
            print(f"  Loaded {name} Q-table from '{q_table_path}'")
        except Exception as e:
            print(f"  WARNING: Could not load {name} Q-table from '{q_table_path}': {e}")
            self.q_table = np.zeros((126, 20))

    def choose_action(self, state, legal_actions):
        """Greedy: always pick the action with the highest Q-value."""
        if not legal_actions:
            return None

        s_idx = STATE_TO_INDEX[state]
        best_action = None
        best_q = -float("inf")

        for action in legal_actions:
            r, c = action
            a_idx = r * 5 + c
            q_val = self.q_table[s_idx, a_idx]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS

zsh

```
(venv) ankit@ameyas-MacBook-Air Do_not_eat_the_Poison_MDP %
```

CreatorRa, 2 hours ago Ln 18, Col 10 Spaces: 4 UTF-8 LF Python

What the Results Tell Us

Three findings that go beyond the numbers

01

Both agents learned near-optimal Chomp strategy

Core Finding

Winning 92–93% against a provably optimal solver means both SARSA and Q-Learning independently discovered the key P-position avoidance strategy that game theorists know exists — but cannot analytically derive for a 4×5 board.

02

The backward pass solved the sparse reward problem

Methodology Validated

Chomp gives 0 reward for every move except the last. Without the backward pass, agents would only learn from their final move. 100% vs Heuristic proves the backward pass correctly propagated win/loss signals through every move in the game.

03

SARSA ≈ Q-Learning on small, fully-explorable state spaces

Theory Confirmed

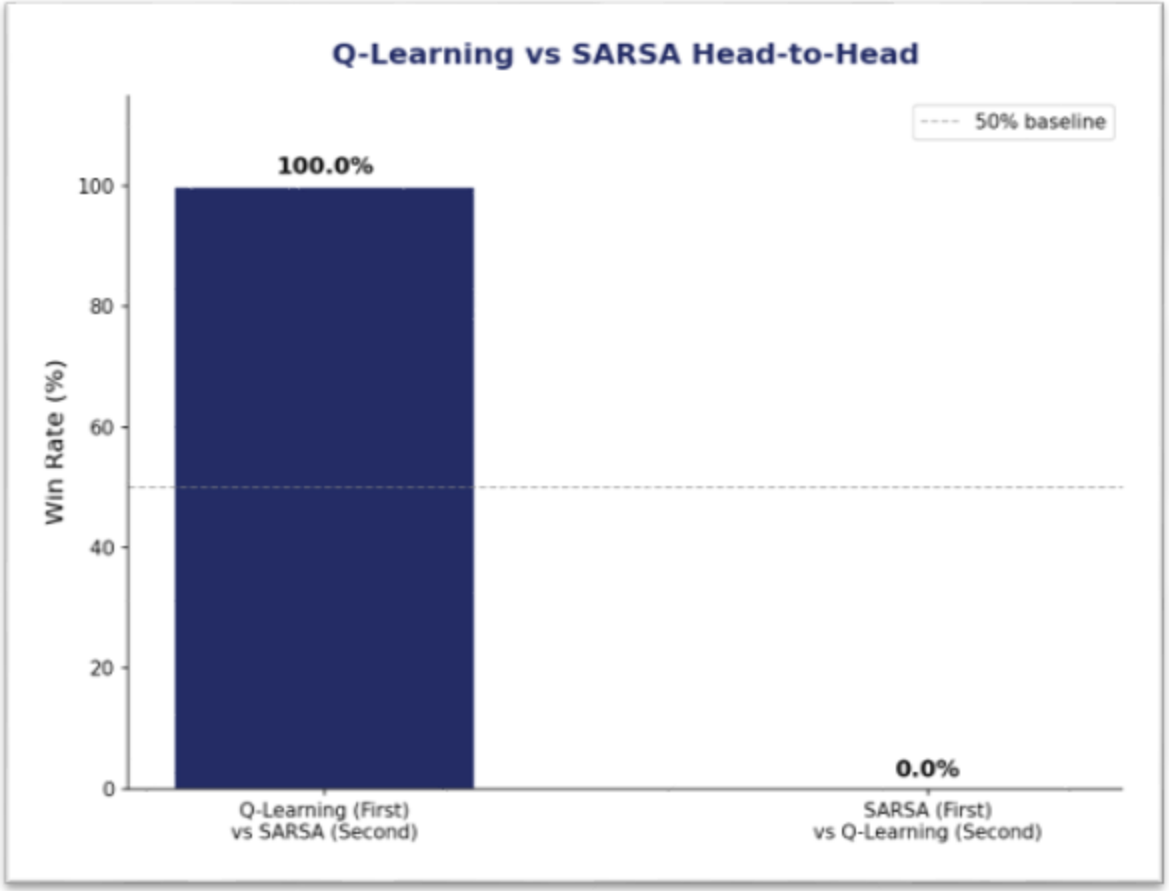
The near-identical results (99.4% vs 99.1%, 92.7% vs 93.4%) confirm the theoretical prediction: on a 126-state MDP with sufficient training, on-policy and off-policy TD methods converge to the same Q-values. The on/off policy distinction matters more on large or stochastic environments.

05

Q - Learning vs SARSA

On-Policy vs. Off-Policy | The Head-to-Head Tournament | A Theoretical Plot Twist

The Ultimate AI Showdown: Q-Learning vs. SARSA



Q-Learning achieved a 100%-win rate against SARSA. Even when granted the mathematical first-mover advantage.

SARSA's on-policy updates penalized it for its own ϵ-greedy exploration mistakes, resulting in a safe but sub-optimal strategy.

In a deterministic, perfect-information game like Chomp, off-policy Q-Learning is fundamentally superior at finding the exact combinatorial game theory solution

Conclusion & Takeaways

What we built, what we proved, what comes next

01

State Space Reduction

Tuple representation collapses 2^{20} = 1,048,576 possible boards to exactly 126 valid states — a 99.988% reduction that makes tabular RL practical.

02

Baselines Defeated

Both agents learned near-optimal strategies, easily beating the Random floor (>99%) and dominating the human Heuristic benchmark (100%), perfectly validating our Backward Pass architecture.

03

On-Policy vs Off-Policy

SARSA (on-policy) and Q-Learning (off-policy) differ in ONE update term: $Q(s',a')$ vs $\max Q(s',\cdot)$. Both use identical interfaces and the same backward pass.

04

Next Steps

Extend to larger board sizes. Deep Reinforcement Learning. Self-Play Curriculum.

Thank you

06

Monte Carlo

Why we chose not to use the Monte Carlo Policy

01

Scientific Control

We needed to isolate the effect of On-Policy vs. Off-Policy learning. SARSA and Q-Learning differ by a single update term, whereas Monte Carlo belongs to a completely different algorithmic family that lacks bootstrapping.

02

The Fatal Flaws (Variance & On-Policy):

Training against a random opponent introduces massive trajectory variance, which MC struggles to handle. Furthermore, standard MC is On-Policy; it would average out the agent's random ϵ-greedy mistakes, resulting in the exact same overly cautious, sub-optimal strategy we saw with SARSA.

03

Our Solution (The Hybrid):

Our Episodic Backward Pass captures the primary benefit of Monte Carlo (waiting for the terminal reward), while our Q-Learning update retains the low-variance, Off-Policy bootstrapping required to aggressively find the mathematical ceiling.

07 Reward Shaping

01

The Reward Hypothesis

The reward signal must communicate strictly *what* we want the agent to achieve (winning), not *how* we want the agent to achieve it

02

The Risk of Sub-Optimal Loops:

If we awarded +0.1 points for every safe bite, the agent's math could prioritize dragging the game out to maximize intermediate points rather than forcing a quick win.

03

The Pure Objective:

By keeping intermediate rewards strictly at 0 and relying entirely on the Episodic Backward Pass, we guaranteed the agent optimized exclusively for the true objective function: defeating the opponent.

08

Constant Learning Rate

Robbins-Monro conditions

01

The Robbins-Monro Trap

Strictly decaying the learning rate guarantees convergence in a *stationary* environment. However, training against an opponent taking random actions makes state transitions highly stochastic from the agent's perspective.

02

The Risk of "Blindness"

If we decayed α to near zero, the agent would lose its ability to update its Q-values if the random opponent suddenly made a rare, edge-case move in the final 1,000 episodes.

03

Recency-Weighted Optimization:

As outlined in Part 2 of our lectures, using a constant step-size parameter ($\alpha=0.1$) acts as an exponential recency-weighted average. This ensures the agent never stops learning and can always overwrite outdated Q-values dynamically.